Fermat’s Little Theorem

A practical guide: why division fails modulo m, brute force, and fast modular inverses

The core problem: modular arithmetic lets us add, subtract, and multiply remainders cleanly—but ordinary division is not always valid. Fermat’s Little Theorem gives a fast way to divide modulo a *prime* number by replacing division with multiplication by an inverse.

Learning path

1. Understand what “mod” means.
2. See why dividing remainders can fail.
3. Find the answer by brute force.
4. Learn modular inverse and Fermat’s Little Theorem.
5. Turn it into an efficient coding pattern.

> Use this guide as a worked example, not as a formula to memorize blindly.

1. The meaning of modulo

“a mod m” is the remainder after dividing integer a by positive integer m. For example, 17 mod 5 = 2 because 17 = 3 × 5 + 2.

In programming, we often keep values modulo a number such as 7, 1,000,000,007, or another prime. This keeps numbers small while preserving the remainder of the final result.

Addition and multiplication behave well

If we know remainders, we can add or multiply and take the modulus again:

```
((a mod m) + (b mod m)) mod m = (a + b) mod m
((a mod m) * (b mod m)) mod m = (a * b) mod m
```

Example with m = 7: (5 × 6) mod 7 = 30 mod 7 = 2. Using remainders first gives (5 × 6) mod 7 = 2 as well.

2. Why ordinary division fails

The tempting but unsafe rule is: “divide the numerator and denominator modulo m.” There is no general rule that lets us divide remainders the way we do with integers.

Counterexample: modulo 7

```
6 / 2 = 3, so (6 / 2) mod 7 = 3
But 6 mod 7 = 6 and 2 mod 7 = 2
Naively dividing the remainders gives 6 / 2 = 3 (this example happens to work).
```

That example does not expose the problem, so consider modulo 6:

```
2 / 2 = 1, so (2 / 2) mod 6 = 1
But 2 mod 6 = 2 and 2 mod 6 = 2
The equation 2*x ≡ 2 (mod 6) has multiple solutions:
x = 1 gives 2*x = 2 ≡ 2
x = 4 gives 2*x = 8 ≡ 2
```

The denominator 2 has no multiplicative inverse modulo 6: there is no integer y for which 2y ≡ 1 (mod 6). So division by 2 modulo 6 is not well-defined. More generally, a denominator b can be divided out modulo m only when gcd(b, m) = 1.

A second trap: integer division before taking mod

Suppose the desired value is (1 / 2) mod 7 in the modular sense. Integer arithmetic says 1 / 2 = 0 if you use integer division, and 0 mod 7 is 0. But modular division is not integer division: the correct modular answer is 4, because 2 × 4 = 8 ≡ 1 (mod 7).

Key distinction

Modular division means multiplying by a modular inverse. It does **not** mean using `/` in code, and it does not mean dividing the remainders directly.

3. Brute force: find the inverse

To calculate a / b modulo m, search for an integer x such that b × x leaves remainder 1 when divided by m. That x is the modular inverse of b.

Example: calculate 1 / 2 modulo 7

```
Try x = 1: (2 * 1) % 7 = 2   -> no
Try x = 2: (2 * 2) % 7 = 4   -> no
Try x = 3: (2 * 3) % 7 = 6   -> no
Try x = 4: (2 * 4) % 7 = 1   -> yes!

So inverse(2) mod 7 = 4
Therefore 1 / 2 mod 7 = 1 * 4 mod 7 = 4
```

General brute-force code (JavaScript)

```
function modInverseBruteForce(b, mod) {
  for (let x = 1; x < mod; x++) {
    if ((b * x) % mod === 1) return x;
  }
  return -1; // no inverse exists
}

console.log(modInverseBruteForce(2, 7)); // 4
```

This takes O(m) trials in the worst case. It is fine for small moduli, but far too slow when m is around 1,000,000,007.

Important condition

An inverse exists exactly when gcd(b, m) = 1. For a prime modulus p, every b not divisible by p has an inverse. If b is divisible by p, the inverse does not exist.

4. Fermat's Little Theorem

Fermat's Little Theorem says: if p is prime and p does not divide a, then

$a^{p-1} \equiv 1 \pmod{p}$

Multiply both sides by the modular inverse of a, or equivalently rearrange the exponent relationship, to obtain:

$a^{p-2} \equiv a^{-1} \pmod{p}$

So the modular inverse of a modulo prime p is a^(p–2) mod p. This turns a search over up to p candidates into a fast exponentiation taking O(log p) multiplications.

Example: inverse of 2 modulo 7

```
2^(7-2) mod 7 = 2^5 mod 7
              = 32 mod 7
              = 4

Check: (2 * 4) % 7 = 1
```

It matches the brute-force answer, but we can compute the power efficiently without calculating an enormous number like 2^1,000,000,005 in full.

5. Fast modular exponentiation

Binary exponentiation repeatedly squares the base and halves the exponent. After every multiplication, take modulo p so the intermediate values stay small.

```
function modPow(base, exp, mod) {
  let result = 1;
  base %= mod;

  while (exp > 0) {
    if (exp % 2 === 1) {
      result = (result * base) % mod;
    }
    base = (base * base) % mod;
    exp = Math.floor(exp / 2);
  }

  return result;
}

function modInverse(a, prime) {
  // Fermat's Little Theorem; requires prime and a % prime !== 0
  return modPow(a, prime - 2, prime);
}

console.log(modInverse(2, 7)); // 4
```

Complexity: O(log p) time and O(1) extra space for this iterative implementation.

6. Full example: compute a / b modulo p

For a modular expression (a / b) mod p, where p is prime and b is not divisible by p:

```
answer = (a mod p) * inverse(b) mod p
       = (a mod p) * b^(p-2) mod p
```

```
const MOD = 1_000_000_007;

function modPow(base, exp, mod) {
  let result = 1;
  base %= mod;
  while (exp > 0) {
    if (exp % 2 === 1) result = (result * base) % mod;
    base = (base * base) % mod;
    exp = Math.floor(exp / 2);
  }
  return result;
}

function divideMod(a, b, mod) {
  const inverseB = modPow(b, mod - 2, mod);
  return ((a % mod) * inverseB) % mod;
}

console.log(divideMod(1, 2, 7)); // 4
```

JavaScript precision warning

JavaScript Number uses floating-point arithmetic and only represents integers exactly up to 2^53 – 1. Multiplying two values close to 1,000,000,007 can exceed that limit, so the multiplication may lose precision. The simple code above is safe for the small example, but is **not generally safe** for arbitrary large modular products in JavaScript Number.

For large moduli in JavaScript, use BigInt (for example, BigInt(base) * BigInt(base) % BigInt(mod)), or a carefully designed multiplication method. BigInt is slower but exact. Do not mix BigInt values with Number values in the same arithmetic expression.

7. Common mistakes checklist

• Using a / b in code instead of multiplying by inverse(b).
• Applying Fermat's theorem when the modulus is composite.
• Forgetting that b must not be divisible by prime p.
• Computing a huge power directly instead of binary exponentiation.
• Assuming Number multiplication is exact for all values under a large modulus.

8. Mental model to remember

Modulo addition/multiplication: reduce the result with %.
Modulo division: find the number that cancels the denominator, i.e. its inverse.
Brute force: try x until (b × x) % p = 1.
Fermat: when p is prime, inverse(b) = b^(p–2) % p.
Fast power: binary exponentiation computes it in O(log p).

Practice questions

1. Find the inverse of 3 modulo 7 by brute force, then using Fermat.
2. Compute (5 / 2) mod 7.
3. Explain why inverse(2) modulo 6 does not exist.
4. What condition must hold for Fermat's Little Theorem inverse formula to work?